Team Lead Fullstack Developer – Technical Assessment

Delivery

Submit a GitHub/GitLab repository containing the application, documentation, tests, and instructions required to run it locally.

You are encouraged to use AI tools, documentation, Stack Overflow, code generators, and any other tools you would normally use in your day-to-day work.

We are not evaluating your ability to memorize framework syntax.

We are evaluating whether you can design, build, explain, debug, and evolve a production-oriented system.

Project: Contact Center Operations Console

Build a small internal application used by customer-support agents to manage customer contacts and their interaction history.

The application consists of:

- Laravel / PHP 8+
- MySQL
- React
- Redux
- TypeScript

You may introduce additional technologies where you believe they are justified.

Functional Requirements

1. Authentication and Roles

Implement authentication.

The system has two roles:

Agent

An agent can:

- View contacts assigned to them
- Search and filter their contacts
- View contact details and interaction history
- Update contact information
- Add interactions

Team Lead / Admin

A team lead can:

- View all contacts
- Perform all agent actions
- Reassign contacts between agents
- Delete or archive contacts

Authorization must be enforced by the backend, not only by the frontend.

---

2. Contacts

A contact contains at minimum:

- id
- name
- email
- phone
- company
- assigned_agent_id
- status
- created_at
- updated_at

Suggested statuses:

- new
- active
- pending
- resolved

The API must support:

- Listing contacts
- Retrieving a contact
- Creating a contact
- Updating a contact
- Deleting or archiving a contact
- Assigning a contact to an agent

---

3. Contact List

The frontend must provide a contacts list.

Filtering and pagination must happen on the server.

Support at minimum:

- Search by name, email, phone, or company
- Filter by status
- Filter by assigned agent
- Sort by creation date
- Sort by most recent interaction
- Pagination

Assume that the production system may eventually contain:

- 250,000+ contacts
- 10,000,000+ interactions

Your implementation does not need to contain that amount of test data, but your database/query design should take the expected scale into consideration.

---

4. Contact Details

The contact details screen should display:

- Contact information
- Assigned agent
- Current status
- Recent interactions

Interactions should appear as a timeline ordered by newest first.

An interaction contains:

- id
- contact_id
- type
- timestamp
- note
- created_by

Example interaction types:

- phone
- email
- sms
- chat

---

5. Interaction Creation

Agents must be able to add interactions to a contact.

Interaction creation should be designed so that an accidental HTTP retry does not create duplicate interactions.

Document how your implementation handles this.

---

6. Concurrent Editing

Assume two agents may open the same contact at approximately the same time.

Your application should have a reasonable strategy for preventing one agent from silently overwriting another agent's changes.

You may choose the implementation.

Document the trade-offs behind your choice.

API Design

Build a RESTful Laravel API.

Use:

- Eloquent
- Form Requests
- Laravel authentication such as Sanctum
- Policies / Gates or an equivalent Laravel authorization mechanism
- API Resources or another consistent response structure
- Proper HTTP status codes
- Centralized error handling where appropriate

You may design the endpoint structure yourself.

We care more about API consistency and reasoning than about a specific URL convention.

Database

Use MySQL.

At minimum your schema should include:

- users
- contacts
- interactions

Create appropriate:

- foreign keys
- indexes

- constraints

Your choices should support the filtering and query patterns required by the application.

Include seeders containing enough realistic sample data to demonstrate the application.

Be prepared to explain why each non-trivial index exists.

Frontend

Use:

- React
- Redux
- TypeScript

The UI does not need to be visually elaborate.

We care about:

- component structure
- state management
- error handling
- loading states
- separation of concerns
- API integration
- maintainability

The application should include:

- Login
- Contacts list
- Contact details
- Add/edit contact
- Interaction creation
- Contact assignment for team leads
- Appropriate confirmation/error handling

Do not assume every API request succeeds.

Redis / Caching

Use Redis for at least one meaningful purpose.

Examples include:

- Caching
- Queues
- Rate limiting
- Distributed locking

Do not add Redis simply to satisfy the requirement.

Document:

1. What you use it for
2. Why you chose that use case
3. When data is invalidated or expires
4. What happens if Redis becomes unavailable

Testing

Tests should cover important behaviour rather than framework boilerplate.

At minimum include backend tests covering:

- Authentication
- Authorization
- Contact creation/update
- Contact filtering
- Interaction creation
- At least one failure or edge case

Include at least one frontend test covering meaningful user behaviour.

We prefer a smaller number of thoughtful tests over a large number of trivial tests.

Docker

Provide a local Docker-based development environment.

A reviewer should be able to clone the repository and start the application with minimal manual setup.

The environment should include the required application services.

Document:

- Startup process
- Database initialization
- Migrations
- Seed data
- Ports
- Environment configuration

CI/CD

Create a GitLab CI pipeline.

At minimum it should:

- Install backend dependencies
- Run backend tests
- Run PHP static analysis or linting
- Install frontend dependencies
- Run frontend tests
- Run TypeScript checking
- Build the frontend

Consider:

- Dependency caching
- Failure behaviour
- Secrets
- Database services required during testing

You do not need to deploy the application to a real production environment.

Production Scenario

Assume this application is successful and six months later has:

- 250,000 contacts
- 10 million interactions
- 100 concurrently active agents
- Approximately 30 API requests per second during peak periods

Add a short document called:

`ARCHITECTURE.md`

Explain what you would review or change before deploying your solution at this scale.

Discuss topics such as:

- Database indexes
- Query behaviour
- Pagination strategy
- Caching
- Queues
- Application scaling
- Database scaling
- Observability
- Failure handling

You do not need to implement every proposed improvement.

We care about your reasoning.

Operational Scenario

Include a short section in `ARCHITECTURE.md` explaining how you would investigate this incident:

> Agents report that the Contacts page became very slow after the interaction table grew to several million rows. Individual contact pages still load normally.

Describe:

- What you would inspect first

- What metrics/logs you would want
- Likely causes
- How you would verify the cause
- Potential fixes
- Risks of those fixes

Engineering Decisions

Create:

`DECISIONS.md`

Document at least five meaningful engineering decisions you made.

For each decision include:

- Problem
- Options considered
- Choice made
- Why
- Trade-offs

Examples might include:

- Pagination strategy
- Authentication approach
- Redux architecture
- Cache invalidation
- Concurrency handling
- Database indexes
- Interaction idempotency
- Error response format

AI Usage

AI tools are explicitly allowed.

Include:

`AI_USAGE.md`

Briefly describe:

- Which AI tools you used
- What you used them for
- Which parts of the generated output you manually reviewed
- At least one AI-generated suggestion you rejected, corrected, or substantially changed
- Why you changed it

There is no penalty for extensive AI usage.

However, during the technical interview you will be expected to explain and modify any part of the submitted solution.

README

Your README should allow another developer to run the project without additional guidance.

Include:

- Prerequisites
- Setup instructions
- Environment configuration
- Commands
- Test commands
- Seed credentials
- Assumptions
- Known limitations